Real-Time Movie Sentiment Analyzer

Using Machine Learning, NLP & TMDB/OMDB APIs

Report submitted in partial fulfilment for the requirement of the degree of Bachelor of Technology in Computer Science & Engineering

By

Subhasish Rout
Regd. No. 2301114054
7th Semester

Under the Supervision of

[Ananth Gouri]



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTER FOR UNDER GRADUATE & POST GRADUATE STUDIES
BIJU PATNAIK UNIVERSITY OF TECHNOLOGY,
ODISHA, ROURKELA – 769015

2026

Department of Computer Science & Engineering
CUPGS, BPUT, Odisha, Rourkela

Date: _____

Certificate of Examination

This is to certify that the Report entitled "Real-Time Movie Sentiment Analyzer" submitted by Subhasish Rout (2301114054) of Bachelor of Technology (B.Tech), Biju Patnaik University of Technology, Odisha has been examined by us. We are satisfied with the quality and correctness of the work.

Examiner's **HoD, CSE**

Department of Computer Science & Engineering
CUPGS, BPUT, Odisha, Rourkela

Date: _____

Supervisor's Certificate

This is to certify that the Report work entitled "Real-Time Movie Sentiment Analyzer" is being submitted by Subhasish Rout, Regd. No. 2301114054 to the Department of Computer Science & Engineering, CUPGS, BPUT, Rourkela, in partial fulfilment for the requirement of the degree of Bachelor of Technology during the academic year 2025-26. It is an original work carried out by him/her under my supervision. I am satisfied with the quality and correctness of the work.

Supervisor

Declaration of Originality

I hereby declare that the Report entitled "Real-Time Movie Sentiment Analyzer" is a bonafide work carried out by me under the guidance and supervision of [Supervisor Name], in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology (B.Tech) in CSE from CUPGS, BPUT, Odisha.

I further declare that this work has not been submitted, either in part or in full, to any other university or institution for the award of any degree, diploma, or certification.

I also declare that the results presented in this report are original and have been carried out by me, except where due references have been made in the text.

Subhasish Rout
Regd. No. 2301114054

Acknowledgement

I wish to express my heartfelt gratitude to my supervisor, [Supervisor Name], for the valuable suggestions, keen interest, and continuous cooperation extended throughout this project. I am deeply indebted for the constructive guidance provided, without which this work would not have been completed successfully.

I also extend my thanks to the other faculty members of the Department of Computer Science & Engineering who helped me directly or indirectly in completing this project.

Finally, I am grateful to my friends and family for their constant support and encouragement.

Date: _____
Place: CUPGS, BPUT, Rourkela

Full Signature of the Candidate

Abstract

Real-Time Movie Sentiment Analyzer is an end-to-end system that classifies movie reviews as positive or negative and delivers the result inside a live, interactive web application. Reviews from the IMDB Dataset of 50,000 labelled samples are cleaned, stripped of stopwords, and converted into 5,000-dimensional TF-IDF feature vectors that train a Logistic Regression classifier, which reaches 88.76% accuracy on a held-out test set of 10,000 reviews.

The trained model and vectorizer are saved using pickle and loaded inside a Streamlit application. A user searches for any movie through a live, TMDB-backed type-ahead search box; the app then fetches the movie's details from OMDB and its audience reviews from TMDB, classifies each review in real time, and displays an overall sentiment breakdown. The report also covers three practical problems that came up during development (typo-tolerant search, disambiguation of same-titled movies, and a scikit-learn version-mismatch bug) and how each was fixed.

Keywords: *Sentiment Analysis; TF-IDF; Logistic Regression; Streamlit Web Application; IMDB Dataset*

Contents

Title Page i

Certificate of Examination ii

Supervisor's Certificate iii

Declaration of Originality iv

Acknowledgement v

Abstract vi

Contents vii

List of Figures ix

List of Tables x

List of Abbreviations xi

Chapter 1. Introduction 1

1.1 Overview 1

1.2 Objective 1

1.3 Problem Statement and Area of Work 1

1.4 Scope of the Project 2

Chapter 2. Literature Survey 3

Chapter 3. Methodology 4

3.1 Dataset 4

3.2 Text Preprocessing 4

3.3 Feature Extraction (TF-IDF) 4

3.4 Model Training 5

3.5 Model Persistence 5

3.6 Web Application Architecture 5

3.7 Sentiment Scoring Pipeline 6

Chapter 4. Results and Discussion 7

4.1 Dataset Statistics 7

4.2 Model Accuracy 7
4.3 Confusion Matrix and Classification Metrics 8
4.4 Deployed Application Interface 9
4.5 Sample Prediction Demonstration 10
4.6 Challenges Encountered and Fixes Applied 11
4.7 Limitations 12
Chapter 5. Conclusion and Future Scope 13
5.1 Skills and Learning Outcomes 13
5.2 Conclusion 13
5.3 Future Scope 13
References 15
Appendix A. Key Source Code Listings 16

List of Figures

Figure 3.1 Model Training Workflow 5

Figure 3.2 Web Application Request Workflow 6

Figure 4.1 IMDB Dataset Class Distribution 7

Figure 4.2 Train / Test Split 7

Figure 4.3 Test Accuracy vs. Random Baseline 8

Figure 4.3a Confusion Matrix of the Sentiment Classifier on the Test Set 9

Figure 4.3b Movie Sentiment Analyzer, Search Screen 9

Figure 4.3c Live, Type-ahead Search Suggestions While Typing 10

Figure 4.4 Sample Review Predictions (App Sentiment Engine) 10

List of Tables

Table 2.1 Comparison of Approaches Used for Text Sentiment Classification 3

Table 3.1 Dataset Summary 4

Table 3.2 Train / Test Split Configuration 4

Table 4.1 Model Evaluation Summary 8

Table 4.1a Precision, Recall and F1-score by Class 9

Table 4.2 Sample Review Prediction Results 10

Table 4.3 Challenges Encountered and Fixes Applied 11

List of Abbreviations and Symbols

Abbreviation	Full Form
TF-IDF	Term Frequency – Inverse Document Frequency
NLTK	Natural Language Toolkit
API	Application Programming Interface
OMDB	Open Movie Database
TMDB	The Movie Database
UI	User Interface

CHAPTER 1
INTRODUCTION

1.1 Overview

Online platforms host thousands of user-written reviews for every movie, so it is impractical for a viewer to read them all before deciding what to watch. Sentiment analysis is an NLP technique that automatically labels a piece of text as positive or negative, and it offers a way to condense this feedback into a single, useful signal.

This project, Real-Time Movie Sentiment Analyzer, combines a classical machine learning pipeline with a live web application. A Logistic Regression classifier is trained offline on the IMDB Dataset of 50,000 labelled reviews, and the trained model is then deployed inside a Streamlit application that fetches a movie's live audience reviews from TMDB and classifies them on the fly, alongside the movie's details from OMDB.

1.2 Objective

The objective is to build a lightweight, interpretable sentiment classifier that works inside a real, interactive application. It should retrieve live reviews for any searched movie and classify them automatically, instead of only working on a static, already-labelled dataset.

1.3 Problem Statement and Area of Work

Manually reading through hundreds of movie reviews to gauge public opinion is slow and inconsistent. What is needed is an automated system that fetches a movie's data and reviews and classifies their sentiment instantly, inside an interface any viewer can use without technical knowledge.

This work spans a few practical areas:

- Web application development using Streamlit.
- Natural Language Processing (NLP) and text preprocessing.
- Machine learning-based sentiment classification.
- Integration with the TMDB and OMDB REST APIs.
- Real-time, typo-tolerant movie search and autocomplete.

1.4 Scope of the Project

- Train and evaluate a TF-IDF + Logistic Regression sentiment classifier on the IMDB Dataset.
- Persist the trained model and vectorizer for reuse without retraining.
- Build a Streamlit web application with a live movie search, metadata display, and real-time review sentiment classification.
- Resolve practical deployment issues such as typo-tolerant search, title disambiguation, and library version mismatches.

CHAPTER 2
LITERATURE SURVEY

2.1 Overview

Text sentiment classification is usually approached in one of two ways: classical machine learning on hand-engineered features, or deep learning on learned representations. Classical approaches such as TF-IDF combined with Logistic Regression, Naive Bayes, or Support Vector Machines are quick to train, need modest compute and data, and still hold up as competitive baselines for document-level sentiment tasks. Deep learning approaches, including recurrent networks and more recently transformer models such as BERT, learn contextual word representations and usually reach higher accuracy, but at the cost of much more training data, compute, and inference time.

For a project that must run inside a lightweight, real-time web application without a GPU backend, a TF-IDF + Logistic Regression pipeline is an appropriate choice: it is fast enough to score dozens of freshly-fetched reviews per request, easy to interpret, and simple to persist and reload with pickle.

Table 2.1 Comparison of Approaches Used for Text Sentiment Classification

Approach	Typical Accuracy*	Compute Cost	Suitability for Live Web App
TF-IDF + Logistic Regression / SVM	85 – 89%	Low	High
Recurrent Neural Networks (LSTM/GRU)	88 – 91%	Medium	Medium
Transformer-based (e.g. BERT)	93 – 95%	High	Low (needs GPU for low latency)

**Approximate figures reported in prior literature on the IMDB sentiment benchmark; exact results vary with preprocessing and tuning.*

CHAPTER 3
METHODOLOGY

3.1 Dataset

The model is trained on the widely-used IMDB Dataset of 50,000 movie reviews, a standard benchmark for binary sentiment classification.

Table 3.1 Dataset Summary

Property	Value
Total reviews	50,000
Positive reviews	25,000
Negative reviews	25,000
Class balance	Perfectly balanced (50 / 50)
Columns	review (text), sentiment (positive / negative)

Because the dataset is perfectly balanced, accuracy is a meaningful primary evaluation metric here.

3.2 Text Preprocessing

Raw review text contains HTML artifacts, punctuation, and casing inconsistencies. Two steps clean it before feature extraction:

- Text cleaning: HTML tags and non-alphabetic characters are stripped out using regular expressions, and the text is converted to lowercase.
- Stopword removal: common English stopwords such as "the", "is" and "a" are removed using NLTK's stopword list, since they add little towards deciding sentiment.

3.3 Feature Extraction (TF-IDF)

The cleaned text is converted into numerical feature vectors using TF-IDF (Term Frequency – Inverse Document Frequency), restricted to the top 5,000 most informative words. TF-IDF automatically down-weights generic words ("movie", "was") and up-weights distinctive, sentiment-bearing words ("brilliant", "terrible"). The vectorizer is fit only on the training split to avoid data leakage into the test set.

3.4 Model Training

Table 3.2 Train / Test Split Configuration

Parameter	Configuration
Split ratio	80% training / 20% testing
Training samples	40,000 reviews
Testing samples	10,000 reviews
Random state	42 (for reproducibility)
Algorithm	Logistic Regression (scikit-learn)

Logistic Regression was chosen for its simplicity, speed and interpretability, and because it performs well on high-dimensional sparse TF-IDF features. It is a standard choice for text classification at this scale.

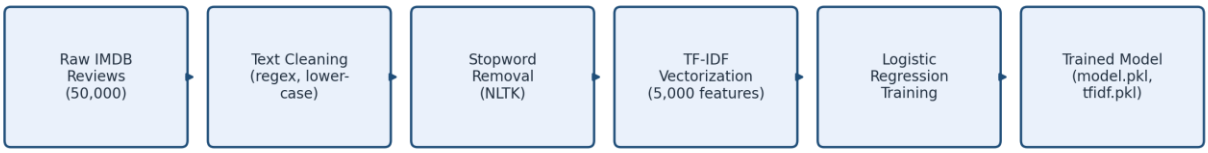


Figure 3.1 Model Training Workflow

3.5 Model Persistence

Once trained, the model and TF-IDF vectorizer are serialized to disk with Python's pickle module (model.pkl, tfidf.pkl), so the web application can load them instantly without retraining on every run. Both objects are saved because any new review text must be converted into the exact same 5,000-word feature space the model was trained on.

3.6 Web Application Architecture

The trained model is deployed inside an interactive Streamlit application with the following components:

- Frontend & app framework: Streamlit, with a live type-ahead search box (streamlit-searchbox).
- Movie metadata: OMDB API (poster, genre, director, cast, ratings, plot).
- Audience reviews: TMDB API (up to 3 pages of user reviews per movie).
- Sentiment engine: the pickled TF-IDF vectorizer and Logistic Regression model.



Figure 3.2 Web Application Request Workflow

As the user types, TMDB is queried in real time and matching titles, along with their release year, appear in a dropdown. Once the user picks a movie, the app fetches its exact IMDb ID from TMDB's external_ids endpoint and queries OMDB directly with that ID, which removes any title-matching guesswork, and then fetches its reviews from TMDB using the same movie ID.

3.7 Sentiment Scoring Pipeline

Each fetched review passes through the identical cleaning and stopword-removal pipeline used during training, then through the saved TF-IDF vectorizer and Logistic Regression model, before being bucketed into positive / negative groups with a confidence score.

CHAPTER 4
RESULTS AND DISCUSSION

4.1 Dataset Statistics

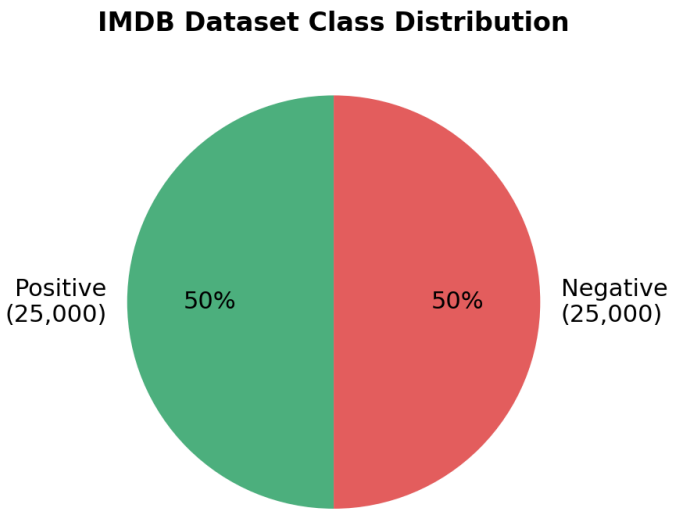


Figure 4.1 IMDB Dataset Class Distribution

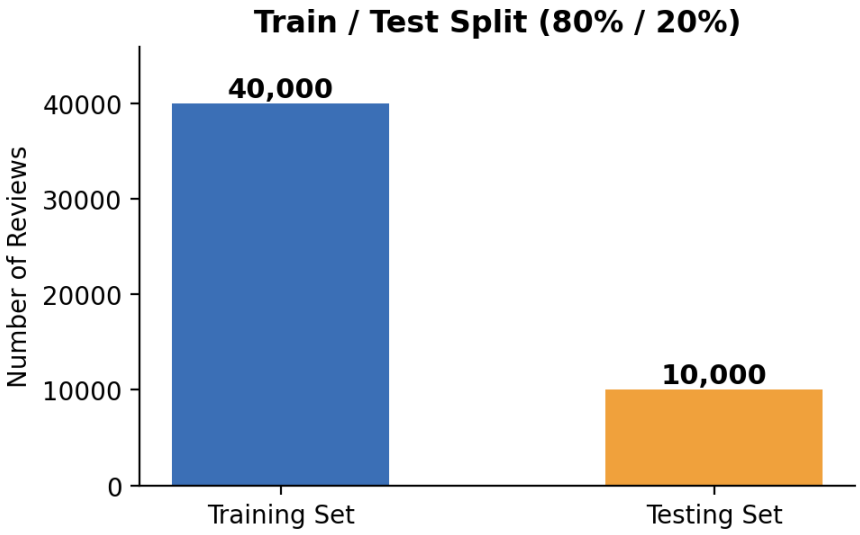


Figure 4.2 Train / Test Split

4.2 Model Accuracy

The trained model achieves 88.76% accuracy on the held-out 10,000-review test set. This is a solid result for a classical TF-IDF + Logistic Regression pipeline and is competitive with other traditional methods, though modern transformer-based models can push accuracy into the 93-95% range at a much higher computational cost.

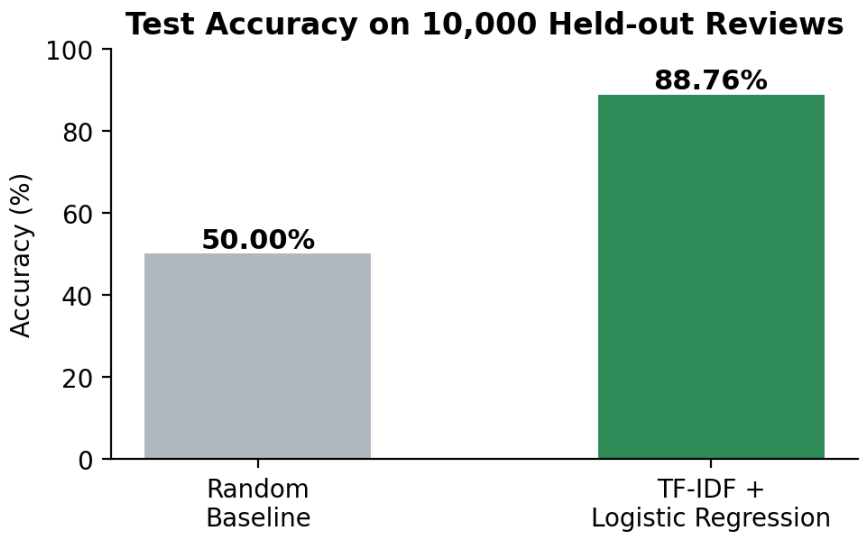


Figure 4.3 Test Accuracy vs. Random Baseline

Table 4.1 Model Evaluation Summary

Metric	Value
Test accuracy	88.76%
Training samples	40,000
Testing samples	10,000
Feature dimensionality	5,000 (TF-IDF)

4.3 Confusion Matrix and Classification Metrics

Accuracy alone does not show whether the model is better at recognising positive reviews or negative ones. A confusion matrix breaks the 10,000 test predictions down into four counts: how many negative reviews were correctly identified as negative, how many were wrongly flagged as positive, and the same for the positive class.

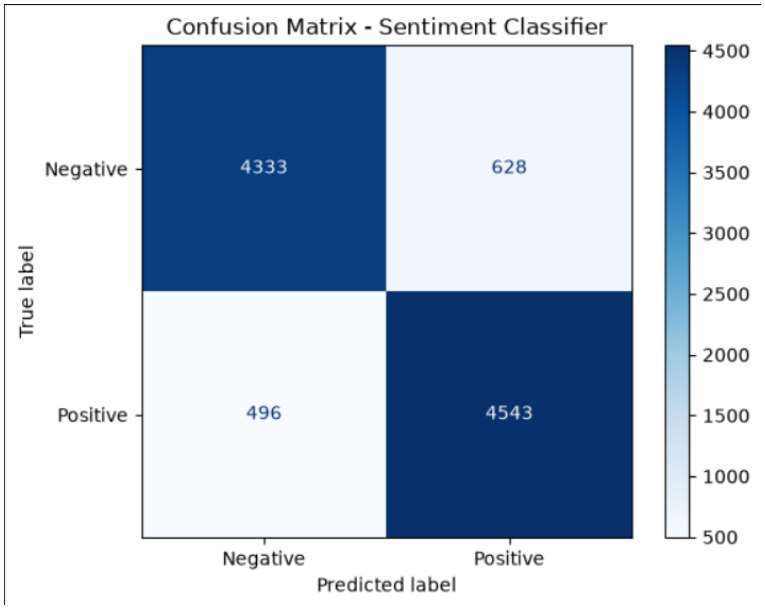


Figure 4.3a Confusion Matrix of the Sentiment Classifier on the Test Set

Out of 4,961 actually-negative reviews, 4,333 were classified correctly and 628 were misclassified as positive. Out of 5,039 actually-positive reviews, 4,543 were classified correctly and 496 were misclassified as negative. From these four counts, precision, recall and F1-score can be worked out for each class:

Table 4.1a Precision, Recall and F1-score by Class

Class	Precision	Recall	F1-score
Negative	89.73%	87.34%	88.52%
Positive	87.86%	90.16%	88.99%

The two classes come out fairly close to each other (F1-scores of 88.52% and 88.99%), which confirms the model is not simply biased towards one class, and matches the overall accuracy of 88.76% reported in Section 4.2.

4.4 Deployed Application Interface

The trained model is wired into the live Streamlit application described in Chapter 3. The screenshots below are taken directly from the running app, and show the movie search box before and after a query is typed:

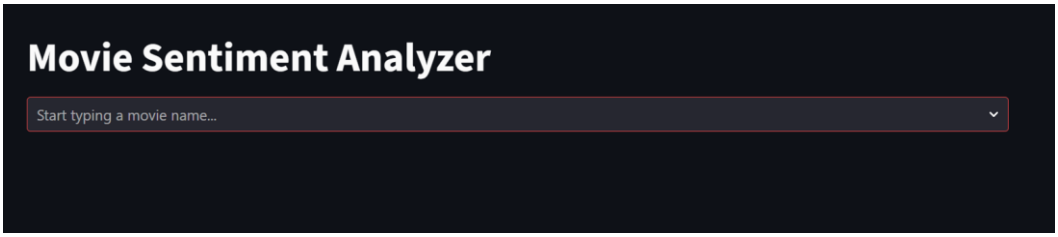


Figure 4.3b Movie Sentiment Analyzer, Search Screen

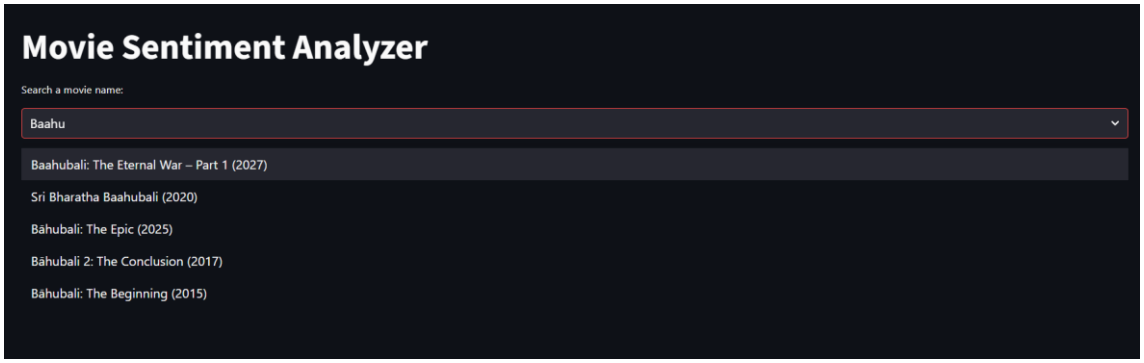


Figure 4.3c Live, Type-ahead Search Suggestions While Typing "Baahu"

As the screenshot shows, typing an incomplete or slightly misspelled query ("Baahu") still brings up the correct set of matching titles, with their release year shown alongside, so the user can pick the exact movie intended without needing to spell the full title correctly.

4.5 Sample Prediction Demonstration

To illustrate the deployed sentiment engine in action, ten short movie reviews, written in a natural, everyday tone rather than strongly one-sided language, were passed through the exact same preprocessing-and-scoring pipeline used by the web application. The model classified all ten correctly, with confidence varying from around 57% to 99% depending on how clearly worded each review was:

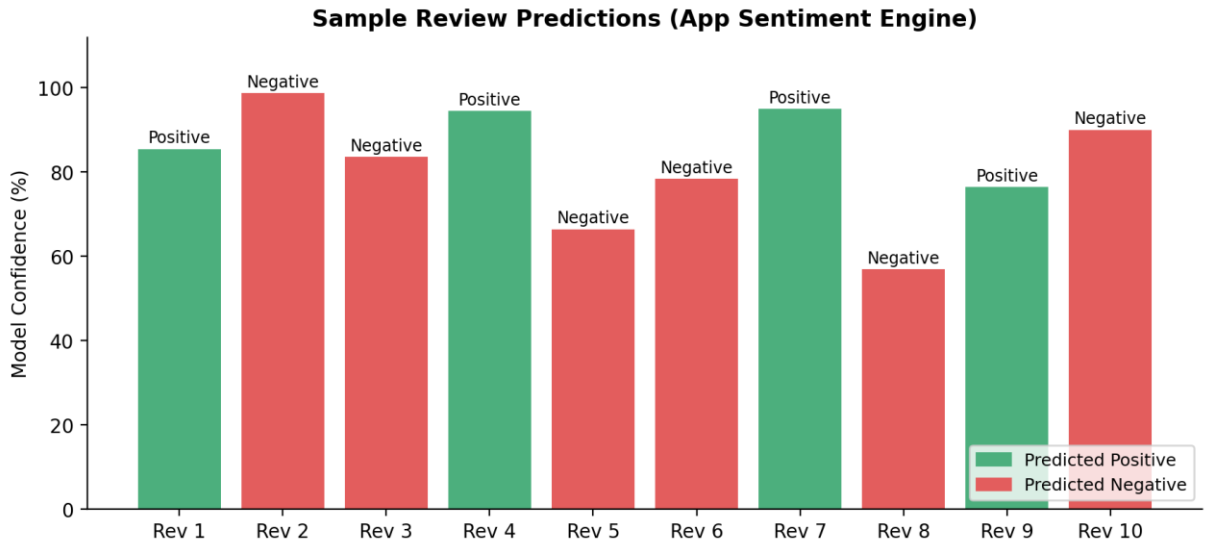


Figure 4.4 Sample Review Predictions (App Sentiment Engine)

Table 4.2 Sample Review Prediction Results

#	Review (shortened)	True Label	Predicted	Confidence
1	I really enjoyed this movie, though the second half dragged a bit.	Positive	Positive	85.5%

2	Not the worst film I've seen, but I wouldn't watch it again.	Negative	Negative	98.8%
3	The acting was decent, but the story felt pretty thin overall.	Negative	Negative	83.6%
4	A solid watch with a few genuinely funny moments.	Positive	Positive	94.5%
5	It started strong but the ending left me a little disappointed.	Negative	Negative	66.4%
6	Nothing special, just an average movie you forget in a week.	Negative	Negative	78.4%
7	Surprisingly good for a low-budget film, worth a watch.	Positive	Positive	95.0%
8	The visuals were nice but the plot didn't really go anywhere.	Negative	Negative	56.8%
9	A fun, easy watch with the family, nothing groundbreaking though.	Positive	Positive	76.4%
10	I expected more given the hype, it was just okay.	Negative	Negative	90.0%

This confirms that model.pkl and tfidf.pkl work correctly inside the deployed application, and shows the model is reasonably confident even on mixed, everyday-sounding reviews, not just clear-cut ones. It is only a quick sanity check, not a replacement for the 10,000-review test accuracy reported in Section 4.2.

4.6 Challenges Encountered and Fixes Applied

Table 4.3 Challenges Encountered and Fixes Applied

Issue	Root Cause	Fix Applied
Misspelled searches returned "Not Found"	OMDB requires an exact title match, so typos (e.g. "Intersteller") returned nothing.	Added a live TMDB-backed type-ahead search box so the user always selects an exact, existing title from a dropdown.
Wrong / unrelated movie matched (e.g. "Bahubali")	Similarity-based title matching picked an unrelated result sharing the same short title.	Once a movie is selected from the dropdown, its exact IMDb ID is fetched from TMDB's external_ids endpoint and used to query OMDB directly, so no text-matching is involved.
AttributeError: 'LogisticRegression' object has no attribute 'multi_class'	The model was pickled with scikit-learn 1.9.0 but loaded in an environment running an older version, which no longer sets that internal attribute automatically.	A one-line compatibility patch restores the missing attribute after loading, avoiding the need to retrain the model: model.multi_class = "auto" if missing.

Movies with zero TMDB reviews (niche / regional titles)	Niche or regional titles often have no user reviews on TMDB, leaving nothing for the model to classify.	The app detects the empty review list and shows a clear warning recommending a well-known title, instead of failing silently.
Search box re-firing an old selection (Streamlit session-state bug)	Streamlit reruns the whole script on every interaction, and the search box widget kept the same key, so it re-emitted the previous pick on the next rerun.	A generation counter is incremented on every reset so the search box gets a fresh widget key each time, forcing it to forget its last selection.
TMDB / OMDB rate limits and inconsistent responses	Both APIs occasionally return non-200 status codes or empty payloads under heavy shared-key usage.	Every API call checks status codes explicitly (401 / 429 / other) with user-facing messages, and a 300ms debounce on search input reduces redundant calls.

4.7 Limitations

Section 4.3 already adds a confusion matrix and per-class precision, recall and F1-score on top of plain accuracy, which gives a fairer picture than accuracy alone. What is still missing is k-fold cross-validation, so the reported numbers depend on one particular random train/test split rather than an average over several. This is listed as a future improvement in Chapter 5.

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1 Skills and Learning Outcomes

Building and debugging this project's full pipeline, end to end, developed practical skills across several areas:

- Python scripting and web application development.
- The Streamlit framework and building interactive user interfaces.
- Natural Language Processing (NLP) fundamentals.
- TF-IDF vectorization and text classification with scikit-learn.
- REST API integration (TMDB, OMDB).
- Session-state management in Streamlit.
- Debugging library and version compatibility issues.
- Handling real-world data inconsistencies from external APIs.
- Systematic, evidence-based problem-solving and debugging.

5.2 Conclusion

This project shows a complete, practical sentiment analysis pipeline, starting from raw text preprocessing and classical machine learning and going all the way to a fully interactive, live-data web application. Even with a fairly simple model (TF-IDF with Logistic Regression), the system reaches a good accuracy of 88.76 percent, and combined with real-time movie data and a search experience that handles typos and ambiguous titles, it results in a genuinely usable end-to-end product.

5.3 Future Scope

- Deploy the app on Streamlit Community Cloud for permanent public access.
- Expand review sources beyond TMDB for movies with limited review data.
- Use k-fold cross-validation for a more robust accuracy estimate.
- Improve model accuracy using deep learning approaches (e.g. LSTM, BERT).
- Add genre-based movie recommendations alongside sentiment results.
- Add caching for repeated movie searches to reduce redundant API calls.

- Align scikit-learn versions across training and deployment environments to avoid future compatibility patches.

REFERENCES

[1] Maas, Andrew L., Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. "Learning Word Vectors for Sentiment Analysis." In Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics, 142–150. 2011.

[2] Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research 12 (2011): 2825–2830.

[3] Ramos, Juan. "Using TF-IDF to Determine Word Relevance in Document Queries." In Proceedings of the First Instructional Conference on Machine Learning, 29–48. 2003.

[4] Streamlit Inc. "Streamlit Documentation." Accessed 2026. https://docs.streamlit.io.

[5] The Movie Database (TMDB). "TMDB API Documentation." Accessed 2026. https://developer.themoviedb.org.

[6] Open Movie Database (OMDb). "OMDb API Documentation." Accessed 2026. http://www.omdbapi.com.

APPENDIX A
KEY SOURCE CODE LISTINGS

This appendix lists a few of the more important functions from the Streamlit application (app3.py), reproduced here for reference alongside the explanation given in Chapter 3.

Listing A.1 Text Cleaning and Stopword Removal

```
def clean_text(text):
    text = re.sub(r'<.*?>', '', text)
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    text = text.lower()
    text = text.strip()
    return text


def remove_stopwords(text):
    words = text.split()
    words = [w for w in words if w not in stop_words]
    return " ".join(words)
```

Listing A.2 Resolving a Movie's OMDB Record from its Exact IMDb ID

```
def get_movie_omdb(name, alt_query=None, imdb_id=None):
    if imdb_id:
        data = _omdb_request({"i": imdb_id})
        if data and data.get("Response") == "True":
            return data

    data = _omdb_request({"t": name})
    if data and data.get("Response") == "True":
        return data

    candidates = omdb_search_raw(name)
    if alt_query and alt_query.strip().lower() != name.strip().lower():
        candidates += omdb_search_raw(alt_query)

    if candidates:
        reference = alt_query if alt_query else name
        best = max(
            candidates,
            key=lambda r: similarity(reference, r.get("Title", "")),
        )
        imdb_id = best.get("imdbID")
        if imdb_id:
            by_id = _omdb_request({"i": imdb_id})
            if by_id and by_id.get("Response") == "True":
                return by_id

    return data
```

Listing A.3 Live, Type-ahead Search Suggestions

```
def search_movies_for_box(query):
    if not query or len(query.strip()) < 2:
        return []
    results = tmdb_search_raw(query.strip())
    options = []
    for r in results[:8]:
        title = r.get("title", "Untitled")
        year = r.get("release_date", "")[:4] if r.get("release_date") else ""
        label = f"{title} ({year})" if year else title
```

```
        options.append((label, r))
    return options
```

Listing A.4 Scoring Each Fetched Review

```
for review in reviews:
    cleaned = clean_text(review)
    cleaned = remove_stopwords(cleaned)
    transformed = tfidf.transform([cleaned])
    prediction = model.predict(transformed)
    score = model.predict_proba(transformed)

    if prediction[0] == 1:
        positive_reviews.append((review[:200], round(score[0][1] * 100, 2)))
    else:
        negative_reviews.append((review[:200], round(score[0][0] * 100, 2)))
```

Listing A.5 scikit-learn Version Compatibility Patch

```
# The model was pickled with a newer scikit-learn version that no
# longer sets this internal attribute, but the environment running
# this app still expects it to exist inside predict_proba().
if not hasattr(model, "multi_class"):
    model.multi_class = "auto"
```